

INFORME INDIVIDUAL DE TRABAJO

José Antonio Taborda Morán · Módulo de Persistencia, Archivos Físicos y Tiempo · Proyecto g_agenda

Asignatura: Programación (IFCD0210 – Desenvolupament d'aplicacions amb tecnologia web)

Grupo: ROSIJO — **Fecha:** 22 de julio de 2026

Naturaleza del documento: *informe individual, subjetivo y en clave de feedback, sobre mi aportación al proyecto.*

1. Mi parte en el proyecto (qué he hecho)

Mi responsabilidad dentro del grupo fue el núcleo de arranque del sistema: todo lo que ocurre antes de que exista una sola tabla en pantalla. Es decir, leer la configuración, cargar los idiomas y resolver los cálculos de calendario que después consumen mis compañeros. En concreto, me encargué de estos cuatro bloques:

- **Clase entidad Configuracion.java:** diseñé el objeto que almacena de forma limpia el año, el mes y los idiomas de entrada y salida. El resto del programa no lee el archivo por su cuenta; le pregunta a este objeto con métodos como `o` .
- **Lector de config.txt (leerConfig):** programé la lectura del archivo plano de configuración de forma tolerante a fallos. Si falta una línea, viene vacía o el año y el mes no son números válidos, el método no revienta: devuelve un `controlado` y el `aborta` con elegancia, en lugar de lanzarle una excepción a la cara al usuario.
- **Motor de diccionarios (cargarDiccionario / getTraduccion):** construí el sistema de internacionalización usando un `.` Cargo el fichero línea a línea, separando el código numérico de su texto por el carácter “;”, e indexo cada etiqueta por su código (001, 002, ...). Todo con lectura en UTF-8 para no corromper acentos ni ñes.
- **Cálculos de fecha estáticos con java.time.LocalDate:** desarrollé `(con los años bisiestos resueltos por la propia API)`, y `,` que calcula el desfase de la primera semana. Son los métodos que permiten que las tablas de Simon dibujen el mes empezando exactamente en el día correcto de la semana.

Esquema representativo de mi módulo. A modo de resumen, así es como se comunican mis tres piezas dentro del arranque del programa: primero se lee la configuración de forma protegida, después se carga el diccionario del idioma y, a partir de ahí, el resto del sistema sólo consulta traducciones y fechas ya resueltas.

```
// 1) Lectura tolerante a fallos de config.txt
Configuracion config = GestionDatosTiempo.leerConfig("config.txt");
if (config == null) return;           // aborta con elegancia, sin excepción

// 2) Carga del diccionario del idioma de salida (UTF-8)
GestionDatosTiempo gestor = new GestionDatosTiempo();
gestor.cargarDiccionario(config.getIdiomaSalida()); // internacional.ENG -> HashMap

// 3) Cálculos de calendario que consumen las tablas
int dias    = GestionDatosTiempo.getDiasDelMes(config.getAnio(), config.getMes());
int desfase = GestionDatosTiempo.getDiaDeLaSemana(config.getAnio(), config.getMes(), 1);
```

Nota: esquema simplificado con fines explicativos; el detalle completo está en las clases y del repositorio.

2. Lo que pulimos y lo más complicado

El doble nivel de parseo del diccionario. Las líneas del fichero de idiomas se separan por “;” (código;valor), pero algunos valores contienen a su vez listas separadas por comas —los siete días de la

semana, los doce meses—. Tuve que dejar clarísimo ese contrato para que el que hace Simon en el frontend no se rompiera. De hecho, el error que más nos costó, un `ArrayIndexOutOfBoundsException`, venía justo de aquí: se pedía la posición [4] de una lista que sólo tenía cuatro elementos (índices 0 a 3). Ajustamos entre los dos el diccionario y las llamadas hasta que los índices cuadraron.

La codificación de caracteres (encoding). Al ejecutar el programa en terminales de distintos sistemas operativos, palabras como “Miércoles” o “Sábado” salían con símbolos corruptos. Forcé la lectura en UTF-8 en todos mis flujos y Simon inyectó la etiqueta `<meta charset="UTF-8">` en la cabecera del HTML. Entre los dos blindamos los acentos en cualquier plataforma.

El desfase del primer día del mes. Conceptualmente fue lo más difícil. Entender que un mes no empieza necesariamente en lunes, y que la primera fila de la tabla necesita celdas vacías antes del día 1 para que el calendario cuadre visualmente, me obligó a pensar la fecha no como un número suelto sino como una posición dentro de una rejilla.

3. Lo que me gustó y lo que me costó

Lo que me gustó. Ver el calendario computarse solo. Que el programa dedujera cuántos días tiene el mes, en qué día de la semana cae el 1 y cuántas semanas hay que dibujar, sin que yo lo escribiera a mano, fue muy satisfactorio. Y sobre todo el sistema de idiomas: cambiar `es` por `en` en el `main` y ver toda la web traducida sin tocar una sola línea de código Java es de las cosas de las que más orgulloso me siento del proyecto.

Lo que me costó. El cambio de mentalidad hacia la “programación tolerante a fallos”. Al principio mi código daba por hecho que el archivo de configuración siempre estaría bien escrito. Aprender a desconfiar de la entrada —líneas vacías, espacios de más, formatos inesperados— y a blindar cada lectura para que el programa no colapse fue un cambio de chip importante y el que más esfuerzo me pidió.

4. Mi aporte al equipo

Fui la primera etapa del flujo de datos: sin mi lectura de `input.txt` y sin mis diccionarios cargados en memoria, ni la lógica de Robert ni el render de Simon tenían un punto de partida. Mi módulo es el cimiento sobre el que se monta todo lo demás.

Además, tener que coordinar los nombres de mis métodos de fecha (`getDay`, `getMonth`) con lo que Simon necesitaba en sus bucles me obligó a pensar en interfaces limpias y estables: un método de fecha que cambie de nombre o de parámetros a mitad de proyecto rompe el código del compañero. Aprendí que en un equipo el contrato entre módulos es tan importante como el código de dentro.

También participé en la depuración conjunta de dos de los tres fallos críticos que documentamos: el del diccionario, que nacía en mi contrato de datos, y el problema de codificación de los acentos, que resolví forzando UTF-8 en mis flujos de lectura mientras Simon reforzaba la cabecera del HTML.

5. Lo que mejoraría si tuviera más tiempo

Con perspectiva, hay dos cosas que puliría. La primera, externalizar todavía más la configuración: hoy el `main` asume un orden fijo de líneas, y me gustaría que aceptara pares clave-valor (`key=value`) para que fuera aún más difícil de romper. La segunda, añadir mensajes de error más informativos cuando falta un archivo de idioma, indicando exactamente qué fichero se esperaba encontrar en la raíz, en lugar de un aborto silencioso. Ninguna de las dos era imprescindible para el enunciado, pero son el siguiente paso natural del módulo.

6. Conclusión

Este proyecto me ha demostrado que programar en equipo no consiste en escribir líneas de código aisladas, sino en construir una pieza en la que otras dos personas van a apoyarse. Pasar de resolver ejercicios sueltos de consola a ser responsable del arranque, la persistencia y el tiempo de una aplicación real —tolerante a fallos, internacionalizada y con un calendario que se calcula solo— ha sido exigente, pero es donde más he aprendido. Me llevo dos lecciones que creo que me acompañarán: desconfiar siempre de los datos de entrada, y cuidar la interfaz que ofrezco a mis compañeros tanto como la lógica interna de mi propio módulo.

Anexo — Mis clases y métodos (referencia para la defensa)

Clase / Método	Responsabilidad
Configuracion.java	Entidad que almacena año, mes e idiomas de entrada/salida.
leerConfig()	Lectura tolerante a fallos de config.txt; devuelve null controlado.
cargarDiccionario()	Carga internacional.XXX en un HashMap con codificación UTF-8.
getTraduccion(cod)	Devuelve la etiqueta de texto asociada a un código (001, 002...).
getDiasDelMes()	Días del mes vía LocalDate, con años bisiestos resueltos.
getDiaDeLaSemana()	Día de la semana en que cae una fecha concreta.
getFilaSemanaMes()	Desfase / fila de semana para colocar el día en la rejilla.